

Práctica: Semilla de datos y consultas LINQ de alto rendimiento

Paso a paso — Poblar roles maestros con HasData, auditar creación y modificación en SaveChangesAsync, y optimizar la API REST con Include, AsNoTracking y Select

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Configurar datos maestros semilla (Seed Data) en EF Core para Roles y Centros de Costo mediante HasData() y aplicar la migración sobre SQL Server.
- Sobrescribir SaveChangesAsync en SolicitudesDbContext para poblar automáticamente los campos de auditoría (FechaCreacion, CreadoPor, etc.) en toda EntidadAuditable.
- Implementar en SolicitudService las operaciones de creación y modificación persistiendo con SaveChangesAsync para activar y validar la auditoría automática.
- Conectar SolicitudService a SolicitudesDbContext, aplicando Eager Loading con Include y ThenInclude para eliminar el antipatrón del N+1 en el endpoint por ID.
- Optimizar las consultas de solo lectura en el endpoint GET de solicitudes mediante AsNoTracking().
- Construir un endpoint con proyección tipada usando Select() para transferir únicamente las columnas requeridas por el cliente en un DTO ligero.
- Configurar el registro del servicio con ciclo de vida Scoped en Program.cs y verificar los endpoints REST en ejecución.

2. Desarrollo paso a paso

Paso 1 — Configurar datos semilla con HasData() en RolConfiguration y CentroCostoConfiguration

Para asegurar que el sistema inicie con los roles y centros de costo requeridos por las reglas del negocio, definimos los datos maestros iniciales usando HasData(). Configuramos cada entidad en su propio archivo dentro de SolicitudesApp.Infrastructure/Persistencia con el código completo a continuación:

Archivo 1: SolicitudesApp.Infrastructure/Persistencia/RolConfiguration.cs — Define la clave primaria, la longitud máxima del nombre y los 4 roles del sistema (Usuario, Revisor, Aprobador, Tesorería):

RolConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infrastructure.Persistencia
{
    public class RolConfiguration : IEntityTypeConfiguration<Rol>
    {
        public void Configure(EntityTypeBuilder<Rol> builder)
        {
            builder.HasKey(r => r.Id);
            builder.Property(r => r.Nombre).IsRequired().HasMaxLength(50);

            builder.HasData(
                new Rol { Id = 1, Nombre = "Usuario" },
                new Rol { Id = 2, Nombre = "Revisor" },
                new Rol { Id = 3, Nombre = "Aprobador" },
                new Rol { Id = 4, Nombre = "Tesoreria" }
            );
        }
    }
}
```

Archivo 2: SolicitudesApp.Infrastructure/Persistencia/CentroCostoConfiguration.cs — Mapea la tabla CentrosCosto, sus propiedades y los 3 centros de costo iniciales de la organización:

CentroCostoConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infrastructure.Persistencia
{
    public class CentroCostoConfiguration : IEntityTypeConfiguration<CentroCosto>
    {
        public void Configure(EntityTypeBuilder<CentroCosto> builder)
        {
            builder.ToTable("CentrosCosto");
            builder.HasKey(c => c.Id);
            builder.Property(c => c.Codigo).IsRequired().HasMaxLength(20);
            builder.Property(c => c.Nombre).IsRequired().HasMaxLength(100);

            builder.HasData(
                new { Id = 1, Codigo = "CC-TI-01", Nombre = "Tecnologías de la Información",
                    Activo = true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" },
                new { Id = 2, Codigo = "CC-OPS-02", Nombre = "Operaciones y Logística", Activo =
                    true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" },
                new { Id = 3, Codigo = "CC-VEN-03", Nombre = "Ventas y Marketing", Activo = true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" }
            );
        }
    }
}
```

Claves primarias explícitas en HasData: EF Core necesita que especifiques el Id en HasData() para determinar si el registro ya existe en la base de datos o si debe generarse un INSERT durante la migración.

Paso 2 — Crear y aplicar la migración en Windows (PowerShell)

Abre una terminal PowerShell en Windows posicionada en la raíz de la solución (C:\Users\...\SolicitudesApp). Dado que las carpetas de los proyectos están directamente en la raíz de la solución (sin subcarpeta src/), ejecuta los comandos indicando directamente los nombres de los proyectos:

PowerShell (Windows) – Ejecutar en la raíz de la solución SolicitudesApp

```
# 1. Crear la migración para los datos semilla:
dotnet ef migrations add MigracionSemilla --project SolicitudesApp.Infrastructure --startup-
project SolicitudesApp.Api

# 2. Aplicar la migración a la base de datos SQL Server:
dotnet ef database update --project SolicitudesApp.Infrastructure --startup-project
SolicitudesApp.Api
```

Solución al error 'Unable to retrieve project metadata': Si recibes el error 'Unable to retrieve project metadata. Ensure it's an SDK-style project', la causa es que dotnet ef no encuentra el proyecto porque la ruta especificada no existe (por ejemplo, usar 'src/SolicitudesApp.Infrastructure' cuando no existe una carpeta 'src/'). En Windows PowerShell, ubícate en la raíz de la solución (donde está el archivo .sln o .slnx) y referencia directamente 'SolicitudesApp.Infrastructure' y 'SolicitudesApp.Api' (o './SolicitudesApp.Infrastructure' / './\SolicitudesApp.Infrastructure').

Paso 3 — Automatizar la auditoría en SaveChangesAsync

Sobrescribimos SaveChangesAsync en SolicitudesDbContext para que toda entidad que herede de EntidadAuditable reciba su fecha UTC automáticamente al ser persistida, evitando asignaciones manuales en los controladores.

SolicitudesDbContext.cs

```
public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
{
    var entradas = ChangeTracker.Entries<EntidadAuditable>();

    foreach (var entrada in entradas)
    {
        if (entrada.State == EntityState.Added)
        {
            entrada.Entity.FechaCreacion = DateTime.UtcNow;
            if (string.IsNullOrEmpty(entrada.Entity.CreadoPor))
                entrada.Entity.CreadoPor = "sistema"; // Se vinculará al usuario autenticado en
S13
        }
        else if (entrada.State == EntityState.Modified)
        {
            entrada.Entity.FechaModificacion = DateTime.UtcNow;
            if (string.IsNullOrEmpty(entrada.Entity.ModificadoPor))
                entrada.Entity.ModificadoPor = "sistema";
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}
```

Auditoría centralizada con ChangeTracker: ChangeTracker.Entries<T>() intercepta todas las entidades modificadas en la transacción actual, asegurando marcas de tiempo homogéneas en UTC y garantizando trazabilidad contable.

Paso 4 — Implementar creación y modificación en SolicitudService con auditoría automática

Para aprovechar la auditoría centralizada configurada en el Paso 3, implementamos las operaciones de escritura en SolicitudService: CrearAsync para registrar una nueva solicitud con sus líneas de detalle y CambiarEstadoAsync para procesar transiciones de estado. Al invocar await db.SaveChangesAsync(), EF Core detecta si las entidades están en estado Added o Modified, asignando automáticamente FechaCreacion, CreadoPor, FechaModificacion y ModificadoPor sin requerir asignaciones manuales en los controladores ni en los servicios.

Archivo 1: Application/Solicitudes/Dtos.cs — Define los contratos DTO de entrada para creación y cambio de estado, y el DTO de respuesta con los campos auditados FechaCreacion y FechaModificacion:

Dtos.cs

```
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Solicitudes;

public record CrearSolicitudItemDto(string Concepto, int Cantidad, decimal PrecioUnitario);

public record CrearSolicitudDto(string Motivo, int SolicitanteId, int? CentroCostoId,
List<CrearSolicitudItemDto> Items);

public record CambiarEstadoDto(EstadoSolicitud Destino, int ResponsableId, string? Comentario);

public record SolicitudRespuestaDto(
    int Id,
    string Motivo,
    decimal MontoTotal,
    string Estado,
    string AprobadorRequerido,
    IEnumerable<AprobacionHistorialDto> Historial,
    DateTime? FechaCreacion = null,
    DateTime? FechaModificacion = null);

public record AprobacionHistorialDto(string Estado, string? Responsable, string? Comentario,
DateTime Fecha);
```

Archivo 2: Application/Solicitudes/SolicitudService.cs — Implementa CrearAsync (activa EntityState.Added) y CambiarEstadoAsync (activa EntityState.Modified), delegando a SaveChangesAsync la asignación automática de auditoría:

SolicitudService.cs

```
using Microsoft.EntityFrameworkCore;
using SolicitudesApp.Application.Aprobaciones;
using SolicitudesApp.Domain.Entidades;
using SolicitudesApp.Infrastructure.Persistencia;

namespace SolicitudesApp.Application.Solicitudes;

public interface ISolicitudService
{
    Task<SolicitudRespuestaDto> CrearAsync(CrearSolicitudDto dto);
    Task<SolicitudRespuestaDto> CambiarEstadoAsync(int id, CambiarEstadoDto dto);
    Task<SolicitudRespuestaDto> ObtenerPorIdAsync(int id);
    Task<IReadOnlyList<SolicitudLecturaDto>> ListarLecturaAsync(EstadoSolicitud? filtroExcluir = null);
    Task<IReadOnlyList<SolicitudResumenDto>> ListarResumenAsync(EstadoSolicitud? estado = null);
}

public class SolicitudService(SolicitudesDbContext db, AsignadorAprobador asignador) : ISolicitudService
{
    // Creación: activa EntityState.Added -> SaveChangesAsync asigna FechaCreacion y CreadoPor
    public async Task<SolicitudRespuestaDto> CrearAsync(CrearSolicitudDto dto)
    {
        var solicitud = new Solicitud
        {
            Motivo = dto.Motivo,
            SolicitanteId = dto.SolicitanteId,
            CentroCostoId = dto.CentroCostoId
        };

        foreach (var item in dto.Items)
        {
            solicitud.Detalles.Add(new SolicitudDetalle
            {
                Concepto = item.Concepto,
                Cantidad = item.Cantidad,
                PrecioUnitario = item.PrecioUnitario
            });
        }

        solicitud.RecalcularTotal();

        db.Solicitudes.Add(solicitud);
        await db.SaveChangesAsync(); // <-- Interceptado por SaveChangesAsync (Paso 3)

        return await ObtenerPorIdAsync(solicitud.Id)
            ?? throw new InvalidOperationException("Error al recuperar la solicitud creada");
    }

    // Modificación: activa EntityState.Modified -> SaveChangesAsync asigna FechaModificacion y ModificadoPor
    public async Task<SolicitudRespuestaDto> CambiarEstadoAsync(int id, CambiarEstadoDto dto)
    {
        var solicitud = await db.Solicitudes
            .Include(s => s.Aprobaciones)
            .FirstOrDefaultAsync(s => s.Id == id)
            ?? throw new KeyNotFoundException($"Solicitud #{id} no encontrada");

        var responsable = await db.Usuarios.FindAsync(dto.ResponsableId)
            ?? throw new ArgumentException($"Responsable con ID {dto.ResponsableId} no existe");

        solicitud.CambiarEstado(dto.Destino, responsable, dto.Comentario);

        await db.SaveChangesAsync(); // <-- Interceptado por SaveChangesAsync (Paso 3)

        return await ObtenerPorIdAsync(solicitud.Id)
            ?? throw new InvalidOperationException("Error al recuperar la solicitud modificada");
    }
}
```

Archivo 3: Api/Controllers/SolicitudesController.cs — Expone los endpoints POST y PUT para registrar solicitudes y cambiar su estado en el flujo:

SolicitudesController.cs

```
using Microsoft.AspNetCore.Mvc;
using SolicitudesApp.Application.Solicitudes;
using SolicitudesApp.Domain.Excepciones;

namespace SolicitudesApp.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class SolicitudesController(ISolicitudService service) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Crear([FromBody] CrearSolicitudDto dto)
    {
        var creada = await service.CrearAsync(dto);
        return CreatedAtAction(nameof(ObtenerPorId), new { id = creada.Id }, creada);
    }

    [HttpPut("{id:int}/estado")]
    public async Task<IActionResult> CambiarEstado(int id, [FromBody] CambiarEstadoDto dto)
    {
        try
        {
            var actualizada = await service.CambiarEstadoAsync(id, dto);
            return Ok(new { mensaje = $"Estado actualizado a {dto.Destino}", solicitud = actualizada });
        }
        catch (ReglaDominioException ex)
        {
            return BadRequest(new { error = ex.Message });
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { error = ex.Message });
        }
    }
}
```

Auditoría en acción con SaveChangesAsync: Al agregar una entidad con `db.Solicitudes.Add()`, EF Core la marca como `EntityState.Added`; al invocar `SaveChangesAsync()`, el código del Paso 3 establece automáticamente `FechaCreacion` en UTC y `CreadoPor`. De igual modo, al modificar una entidad rastreada (al cambiar su estado y registrar la aprobación), el Change Tracker detecta la mutación asignando `EntityState.Modified`, activando la asignación automática de `FechaModificacion` y `ModificadoPor`.

Paso 5 — Conectar SolicitudService y Eager Loading (Include)

Implementamos `ObtenerPorIdAsync` en `SolicitudService` cargando la solicitud junto a su solicitante, centro de costo, detalles y aprobaciones en un único viaje a la BD mediante JOINS explícitos generados por `Include` y `ThenInclude`, eliminando el antipatrón del N+1:

Archivo 1: `Application/Solicitudes/SolicitudService.cs` — Consulta por ID con Eager Loading (`Include/ThenInclude`) y mapeo que incluye las marcas de auditoría:

SolicitudService.cs

```
// Application/Solicitudes/SolicitudService.cs: Consulta por ID y mapeo con auditoría
public async Task<SolicitudRespuestaDto?> ObtenerPorIdAsync(int id)
{
    // Eager Loading: genera JOINS explícitos en lugar de múltiples consultas separadas
    var solicitud = await db.Solicitudes
        .Include(s => s.Solicitante)
        .Include(s => s.CentroCosto)
        .Include(s => s.Detalles)
        .Include(s => s.Aprobaciones)
        .ThenInclude(a => a.Responsable)
        .FirstOrDefaultAsync(s => s.Id == id);

    return solicitud is not null ? Mapear(solicitud) : null;
}

private SolicitudRespuestaDto Mapear(Solicitud s) => new(
    s.Id, s.Motivo, s.MontoTotal, s.Estado.ToString(),
    asignador.Determinar(s.MontoTotal),
    s.Aprobaciones.Select(a => new AprobacionHistorialDto(
        a.EstadoResultante.ToString(), a.Responsable?.Nombre, a.Comentario, a.Fecha)),
    s.FechaCreacion,
    s.FechaModificacion
);
```

Archivo 2: Api/Controllers/SolicitudesController.cs — Endpoint GET por ID asíncrono que consume ObtenerPorIdAsync:

SolicitudesController.cs

```
// Api/Controllers/SolicitudesController.cs: endpoint GET por ID asíncrono
[HttpGet("{id:int}")]
public async Task<IActionResult> ObtenerPorId(int id)
{
    var solicitud = await service.ObtenerPorIdAsync(id);
    return solicitud is not null
        ? Ok(solicitud)
        : NotFound(new { mensaje = $"Solicitud #{id} no existe" });
}
```

El peligro del antipatrón N+1: sin Include, consultar 100 solicitudes e iterar sus detalles dispararía 101 consultas SQL independientes (1 + 100). Include() genera un único JOIN relacional que resuelve la jerarquía en un solo viaje de red.

Paso 6 — Optimización de solo lectura con AsNoTracking() en la API REST

Para listar solicitudes en un endpoint GET masivo, apagamos el Change Tracker de EF Core con AsNoTracking() para ahorrar memoria RAM y tiempo de CPU, devolviendo las solicitudes ordenadas por fecha reciente.

Archivo 1: Application/Solicitudes/SolicitudService.cs — Implementa ListarLecturaAsync con AsNoTracking():

SolicitudService.cs

```
// Application/Solicitudes/SolicitudService.cs
public async Task<IReadOnlyList<SolicitudLecturaDto>> ListarLecturaAsync(EstadoSolicitud?
filtroExcluir = null)
{
    // AsNoTracking: hasta un 60% más rápido para consultas de solo lectura
    var consulta = db.Solicitudes
        .AsNoTracking()
        .Include(s => s.Solicitante)
        .Include(s => s.Detalles)
        .AsQueryable();

    if (filtroExcluir.HasValue)
        consulta = consulta.Where(s => s.Estado != filtroExcluir.Value);

    return await consulta
        .OrderByDescending(s => s.FechaCreacion)
        .Select(s => new SolicitudLecturaDto(s.Id, s.Motivo, s.MontoTotal, s.Estado.ToString(),
s.Solicitante!.Nombre, s.FechaCreacion, s.Detalles.Count))
        .ToListAsync();
}
```

Archivo 2: Api/Controllers/SolicitudesController.cs — Endpoint GET general optimizado:

SolicitudesController.cs

```
// Api/Controllers/SolicitudesController.cs: endpoint GET general optimizado
[HttpGet]
public async Task<IActionResult> ListarLectura([FromQuery] EstadoSolicitud? excluirEstado = null)
{
    var listado = await service.ListarLecturaAsync(excluirEstado);
    return Ok(listado);
}
```

Rendimiento con AsNoTracking: úsalo siempre en consultas de lectura donde no se invoque SaveChangesAsync(). Evita que EF Core mantenga copias de comparación de estado en memoria, reduciendo la presión sobre el Garbage Collector.

Paso 7 — Proyección con Select() en el endpoint de resumen

Definimos un DTO ligero y proyectamos con Select() para generar un SELECT en SQL únicamente con las columnas requeridas, transfiriendo solo los datos indispensables para vistas de resumen o tablas.

Archivo 1: Application/Solicitudes/Dtos.cs — DTO proyectado ligero para consultas de resumen:

Dtos.cs

```
// Application/Solicitudes/Dtos.cs: DTO proyectado ligero
public record SolicitudResumenDto(int Id, string Motivo, decimal MontoTotal, string Solicitante,
int TotalItems, string Estado);
```

Archivo 2: Application/Solicitudes/SolicitudService.cs — Implementa ListarResumenAsync proyectando directamente con Select():

SolicitudService.cs

```
// Application/Solicitudes/SolicitudService.cs
public async Task<IReadOnlyList<SolicitudResumenDto>> ListarResumenAsync(EstadoSolicitud? estado
= null)
{
    var consulta = db.Solicitudes.AsNoTracking();
    if (estado.HasValue)
        consulta = consulta.Where(s => s.Estado == estado.Value);

    return await consulta
        .Select(s => new SolicitudResumenDto(
            s.Id, s.Motivo, s.MontoTotal,
            s.Solicitante != null ? s.Solicitante.Nombre : "",
            s.Detalles.Count, s.Estado.ToString()))
        .ToListAsync();
}
```

Archivo 3: Api/Controllers/SolicitudesController.cs — Endpoint GET /api/solicitudes/resumen:

SolicitudesController.cs

```
// Api/Controllers/SolicitudesController.cs: endpoint GET /api/solicitudes/resumen
[HttpGet("resumen")]
public async Task<IActionResult> ListarResumen([FromQuery] EstadoSolicitud? estado = null)
{
    var resumen = await service.ListarResumenAsync(estado);
    return Ok(resumen);
}
```

Proyección a nivel de base de datos: a diferencia de mapear objetos en memoria con LINQ to Objects, Select() sobre IQueryable se traduce directamente a columnas SQL (SELECT s.Id, s.Motivo, ...), ahorrando ancho de banda de red y tiempo de deserialización.

Paso 8 — Registro Scoped en Program.cs y prueba integral de la API REST

Actualizamos el registro de dependencias en Program.cs configurando ISolicitudService con ciclo de vida Scoped (ya que consume SolicitudesDbContext) y probamos el flujo completo de creación, modificación y consultas optimizadas desde Swagger o cliente HTTP:

Archivo 1: Api/Program.cs — Registro Scoped en el contenedor de dependencias:

Program.cs

```
// Api/Program.cs: registro Scoped en el contenedor de dependencias
builder.Services.AddScoped<ISolicitudService, SolicitudService>();
```

Archivo 2: SolicitudesApp.Api.http — Pruebas integrales de todos los endpoints de la API REST:

SolicitudesApp.Api.http

```
### 1. Crear solicitud (POST - Verifica auditoría: FechaCreacion asignada por SaveChangesAsync)
POST http://localhost:5072/api/solicitudes
Content-Type: application/json

{
  "motivo": "Renovación de laptops y monitores",
  "solicitanteId": 1,
  "centroCostoId": 1,
  "items": [
    { "concepto": "Laptop Core i7 32GB", "cantidad": 1, "precioUnitario": 4500.00 },
    { "concepto": "Monitor 27 pulgadas 4K", "cantidad": 2, "precioUnitario": 1200.00 }
  ]
}

### 2. Modificar estado (PUT - Verifica auditoría: FechaModificacion asignada por SaveChangesAsync)
PUT http://localhost:5072/api/solicitudes/1/estado
Content-Type: application/json

{
  "destino": "Enviada",
  "responsableId": 1,
  "comentario": "Solicitud enviada para evaluación técnica"
}

### 3. Obtener solicitud con Eager Loading (Include)
GET http://localhost:5072/api/solicitudes/1
Accept: application/json

### 4. Listado de solo lectura optimizado (AsNoTracking)
GET http://localhost:5072/api/solicitudes?excluirEstado=Pagada
Accept: application/json

### 5. Listado proyectado con Select()
GET http://localhost:5072/api/solicitudes/resumen
Accept: application/json
```

Regla de ciclo de vida en DI: nunca registres como Singleton un servicio que depende de un recurso Scoped como DbContext. Hacerlo genera una 'dependencia cautiva' que causará excepciones en concurrencia y fugas de memoria.

3. Retos adicionales

1. Escribe un método o endpoint auxiliar que mida la diferencia en milisegundos entre una consulta con tracking y otra con AsNoTracking().
2. Usa el método `db.Solicitudes.AsNoTracking().Where(...).ToQueryString()` para inspeccionar e imprimir en consola la sentencia SQL exacta generada por EF Core.
3. Implementa una transacción atómica con `using var tx = await db.Database.BeginTransactionAsync()` para insertar una solicitud con sus detalles en un endpoint POST `/api/solicitudes`.
4. Genera el script T-SQL de la migración de semilla con `dotnet ef migrations script MigracionInicial MigracionSemilla --output migracion-semilla.sql`.
5. Implementa un método para anulación de solicitudes aplicando Soft Delete (marcando un campo `IsDeleted = true`) y comprueba cómo `SaveChangesAsync` actualiza `FechaModificacion` automáticamente sin eliminar el registro físico.

4. Entregables

- Base de datos SQL Server con los roles y centros de costo iniciales poblados mediante la migración MigracionSemilla.
- SolicitudesDbContext.cs con la sobrecarga de SaveChangesAsync asignando automáticamente FechaCreacion y CreadoPor a las entidades auditable.
- SolicitudService.cs y SolicitudesController.cs con operaciones de creación (POST) y modificación (PUT) validando la auditoría automática.
- Implementación de consultas de alto rendimiento con Include, AsNoTracking y proyección tipada con Select.
- Program.cs configurado con AddScoped<ISolicitudService, SolicitudService>() evitando dependencias cautivas.
- Respuestas HTTP en Swagger o cliente REST (.http) demostrando la ejecución de los endpoints de creación, modificación y consultas optimizadas.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 4 · Docente: Edwin Barrientos · New Horizons